Considering an Information-Theoretic Foundation to Question of Generalization in Neural Networks

Aaron Geelon So

***Abstract*—In this project, we replicate a miniature version of Zhang, et. al. 2016 *Understanding Deep Learning Requires Rethinking Generalization.*[1] The impetus for their claim: neural networks can fit random data easily. In this paper, we also aim to provide some more intuition for why this may be problematic to the theory of generalization. Then, after some discussion on our own scaled-down model, reconsider the literature to find other theoretical basis to explain generalizability of neural networks.**

I. Introduction

A. Review of Literature

Why do neural networks work so well? This is one of the most fundamental theory question of learning theory. To get a sense of the 'unreasonable effectiveness' of neural networks, just consider the CIFAR-10 image classification problem. Here, we have images of size 32×32 pixels, where each pixel is a combination of 3 channels, each taking on 255 possible values. It follows that the total number of images is:

$$N = (3 \times 255)^{32 \times 32}.$$

Then, if we want to classify these images using 10 categories, there are 10^N possible ways of classifications. It seems that learning one specific classification out of $10^{(3\times 255)^{32\times 32}}$ should pretty much be impossible.

The unreasonable effectiveness therefore is two-fold: (1) how are neural networks able to achieve such high accuracy, and (2) how is it possible that the representation of the neural network is so compact? In particular, given a collection of 10^N possible classification, information theory tells us that we need at least $O(N)$ bits to represent any single one (and N here is still huge).

This information-theoretic lower bound forces us to consider not *every* single possible classification, but a much smaller subset—by their architecture, neural networks have restricted *capacity*; regularization further reduces capacity by design.

And actually, information theory provides us with a bit more insight: the (expected) number of bits required to represent a model corresponds with the entropy of the *prior distribution* over the hypothesis class $\mathcal{H}$.[4] Restricting capacity is equivalent to adjusting the prior to reduce the entropy. The fact that neural networks are so small (compared to N) indicate that they must have a prior whose probability mass is concentrated over a relatively small collection of hypotheses in $\mathcal{H}$.

Thus, the amazing empirical fact is that neural networks attain such high accuracy: this implies that the prior distribution happens to correspond well with the true distribution (that is, the KL-divergence of the true distribution with respect to the prior is not too large). Finally, this implies that *neural networks should fail to model truly uniform random data.*

We see now why the most important claim of [1] is that *neural networks easily fit random labels.* Stated differently, it is worrying that neural networks can easily fit random training data because this suggests that perhaps the prior distribution isn't as close to the true distribution as we thought; there may be cases where neural networks fail to generalize from true training data.

The authors of [1] discovered this by attempting to train neural networks on real-world data with (i) original labels, (ii) shuffled labels, and (iii) random labels. In all cases, they were able to achieve zero or near-zero training error.

B. Description of Problem

The goal of this project was to replicate some simplified results of [1]. In particular, while it may be troubling that neural networks are able to fit even random training data, that alone should not be too shocking. After all, the training sample size is much smaller than the size of the total space. So it may be reasonable that neural networks have VC dimension on the order of magnitude of our training samples (and so it may realize any possible labeling of data of small enough size).

However, what was unexpected in [1] was the claim that the rate of convergence seemed unaffected by whether the neural network was given true or random data. It seems that this should not be the case, for intuitively, because it is hard to train random data in general, it should be hard to train random data even for a small training size.

The goal of this project therefore is test out this result—what is the training curve of a neural network that is able to achieve near-zero error rate on different sets of data. If indeed neural networks are able to train as easily on random data as it is on true data, then our theory has a long way to go to catch up to the empirical results.

C. Flowcharts and Block Diagrams

The architecture of the neural network used here is as follows:

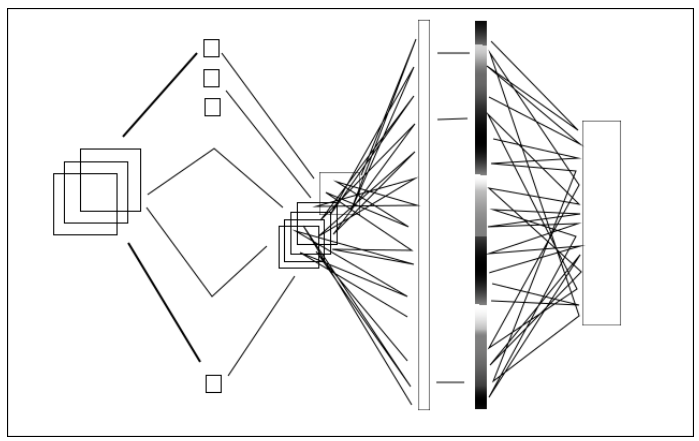

Fig. 1. Input layer is $(32 \times 32 \times 3)$, followed by a convolutional layer with 5×5 kernels, 32 filters, followed by max pooling to reduce each dimension by a factor of 2, followed by a fully connected layer to 1024 features, masked with a 0.9 probability dropout, before termininating with a fully connected layer to 10 features.

Written out as a series of transformations:

$$\begin{aligned} 32 \times 32 \times 3 &\to \text{conv}(5 \times 5 \text{ kernel}) \times 32 \text{ filters} \\ &\to \text{max pooling, reducing dimension by 2} \\ &\to \text{fully connected layer: 1024 features} \\ &\to \text{dropout regularization} \\ &\to \text{fully connected layer: 10 features output} \end{aligned}$$

The advantages of this neural network is that it has relatively few parameters, so that it is much faster to test out different types of data and labels. This turned out to be a pretty important feature for this project, due to time and resources constraints. Additionally, if our main goal is to study the

behavior of neural networks themselves, and less so trying to predict real data well, then a simpler model is conceptually easier to reason about.

On the other hand, with so few parameters, it becomes harder to obtain a high training accuracy, and as a result, we needed to restrict the training data in addition to reusing data. And of course, in general, this makes the model very prone to overfitting the data (although in this study, overfitting is one of the outcomes we want to show—that is, poor generalization).

Prior to training, we also performed some standard preprocessing steps. Our code allowed for zero-centering, normalization, and adding a grayscale channel.

Furthermore, we performed the following data/label transformations. Like the original paper, we included (i) original data and labels, (ii) original data with permuted labels, (iii) original data with random labels and (iv) random data/static with random labels. In the results below, we also demonstrate an example of large training data of static and large training data of original data.

D. Figures

Figure 2 displays an example of an output from running the neural network to try to obtain near-zero accuracy on training. For more figures, see the Appendix.

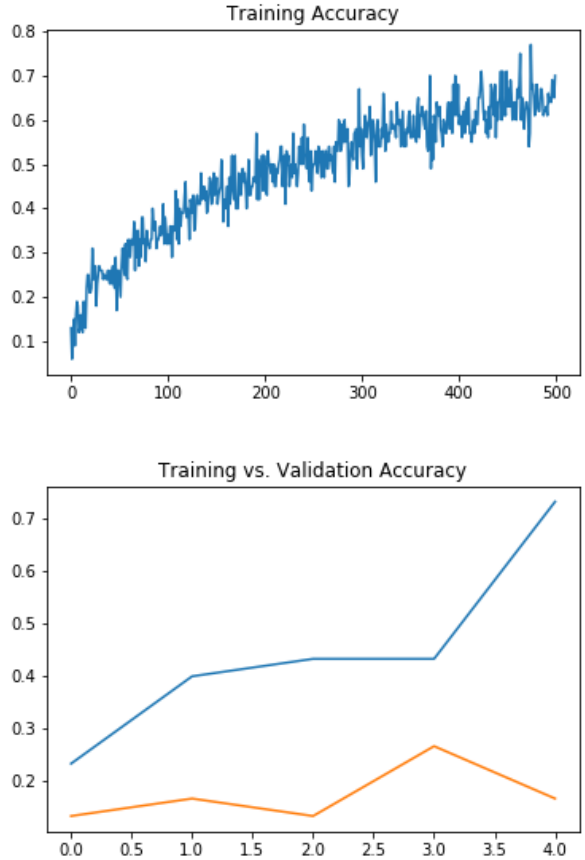

Fig. 2. The training curve on original CIFAR-10 training data (top). A comparison of the training and validation accuracy (bottom).

The main results figure is Figure 3, which shows the relative training rates. We will discuss these results below.

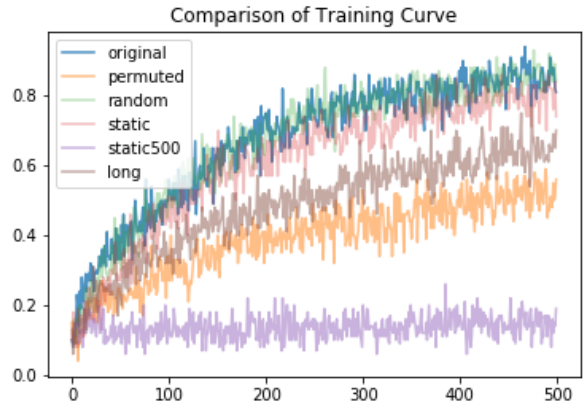

Fig. 3. A comparison of the training curves for across (i) original data, (ii) permuted labels, (iii) random labels, (iv) random data/static, (v) large sample of static, and (vi) large sample of original (trained for a long time).

II. Results

Table I shows the summary of the results. The main comparison to make are between *original*, *permuted*, *random*, and *static*. These were all trained with the same hyperparameters. Oddly, out of these four, permuted labels seem the slowest to converge. But the performance of original, random labels, and random data all seem to exhibit the same convergence rates, each achieving near 90% accuracy on the training set (and the test accuracy is near 10%, corresponding to 10 categories).

	Train Acc.	Test Acc.	Train Time
Original	0.87	0.10	1671 s
Permuted	0.63	0.07	1677 s
Random	0.90	0.27	1666 s
Static	0.87	0.10	1691 s
Static 500	0.17	0.17	1691 s
Original long	0.73	0.17	9668 s

TABLE I
Summary of results.

A. Effort in Optimizing Results

Notice that we included additionally two other examples, *static 500* and *original long*. Original long trained in the whole training set contained in CIFAR-10, and required a little over 2.5 hours to train. As this seemed impractical to run at least 6 times, one solution was to reduce the number of iterations—this required a complementary reduction in the number of training points that the neural networks sees, if we are attempting to attain near-zero error.

The static 500 gives an example where having too many images of static led to nonconvergence; the accuracy stayed around the expected value for random chance. This may reflect a lowered VC dimension as a result of having relatively few parameters.

III. Discussion

A. Discussion of Results

It seemed that the main result reflected what [1] found: the rate of convergence did not seem to be significantly affected by random labels or random data. As noted above, it was

surprising that the permuted label performed so much worse than the other cases. However, as this is one trial, it is hard to say something very statistically significant.

It may be the case that the random seeding led to exceptionally poor performance on the permuted labels or exceptionally good performance on the random data. After all, there are many parameters to consider here. It is not clear how to balance the tradeoff between few parameters and few training samples. Ideally we can minimize the parameters while maximizing the number of training samples; those are the edge cases where the empirical differences in rates of convergence may be most apparent between true data and randomized data.

A future continuation of this work should include most importantly, (i) more trials, (ii) experimentation with number of training points/parameters, and (iii) varying regularization. In particular, at this moment, we only utilized dropout to help regularize, at a fixed value of 0.9. Of course, one of the biggest challenges with trying to understand the behaviors of neural networks is that there are so many possible hyperparameters to change, not to mention network architecture. It seems that perhaps the next step is to consider the problem more theoretically, once again, now that we have some more physical intuition.

B. Comparison with other Papers

The publication of [1] led to a huge amount of response from the learning community. It is, after all, an important theoretical question. More recent literature seems to help explain the results from [1], or even find contradictory results.

One paper repeated more experiments, and found that in some cases there was an empirical difference between random data and true data.[2] It seemed that random data could have a harder time converging, especially with proper regularization such as dropout. Furthermore, one interesting result that they note in their abstract is that "[neural networks] tend to prioritize learning simple patterns first." One line of question worth pursuing then would be to understand how are simple patterns realized within the gradient? In other words, this empirical result seems to suggest that the direction of steepest descent corresponds well with finding simple patterns. What notions of complexity can be properly adapted to account for this?

Another paper looked to explain the results of [1] in another way, inspired by physics and physical systems. They point out that truly random data is almost impossible to generate; most methods of producing pseudorandom data depends on an iterative process. But they suggest that most real world data is generated in a somewhat self-similar or iterative process. In some sense, the chaos that we observe, say in a dynamical system, is not the same as disorder; given enough time and computing power, we may be able to see the patterns in the data. Thus, they are not convinced that the original paper truly was able to show that neural networks succeeded in training on random data and labels.

C. Discussion of Problems Faced

Part of the difficulty in pursuing this problem was the huge amount of choice into what experiment to run. Of course, this is like any sort of neural network problem, where there is a plethora of possible hyperparameters one can utilize. However, as our goal is not just in prediction, but in understanding the behavior of neural networks, it seems that we should spend more time designing specific hypotheses to test and disprove. That is, find a more guided approach to understanding the theoretical properties of neural networks. It does not seem that the data we generated can prove or disprove anything. It just somewhat maintained the plausibility of what [1] stated. However, through this process, there was a lot to be learned.

IV. Acknowledgments

Thank you to Professor Kostic, Jingyu Qian, and the many TAs who have taught and graded my work.

V. References

[1] Zhang, Chiyuan, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. "Understanding deep learning requires rethinking generalization." arXiv preprint arXiv:1611.03530 (2016).

[2] Arpit, Devansh, Stanisław Jastrzęski, Nicolas Ballas, David Krueger, Emmanuel Bengio, Maxinder S. Kanwal, Tegan Maharaj et al. "A closer look at memorization in deep networks." arXiv preprint arXiv:1706.05394 (2017).

[3] Lin, Henry W., Max Tegmark, and David Rolnick. "Why does deep and cheap learning work so well?." Journal of Statistical Physics 168, no. 6 (2017): 1223-1247.

[4] Yang, Liu, Steve Hanneke, and Jaime Carbonell. "Bayesian active learning using arbitrary binary valued queries." In International Conference on Algorithmic Learning Theory, pp. 50-58. Springer, Berlin, Heidelberg, 2010.

Appendix

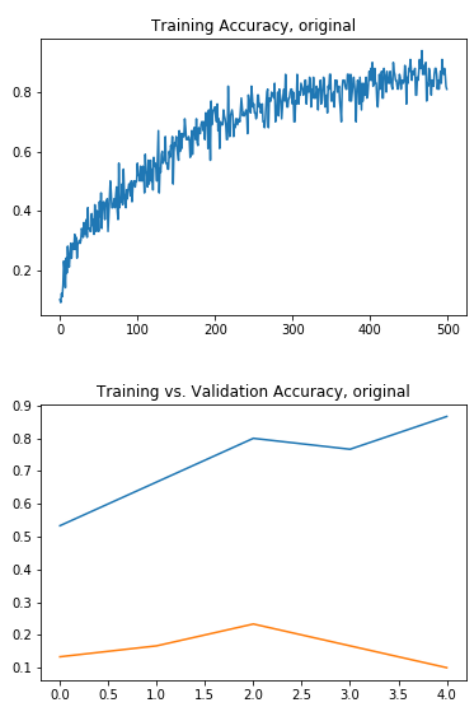

Fig. 4. Original Data and Original Labels

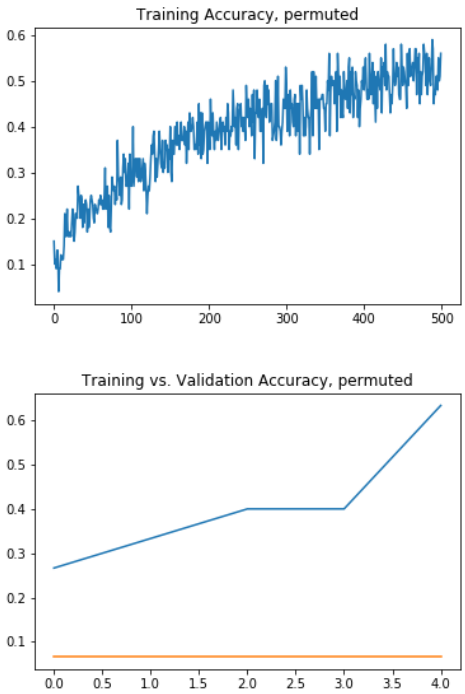

Fig. 5. Permuted Label

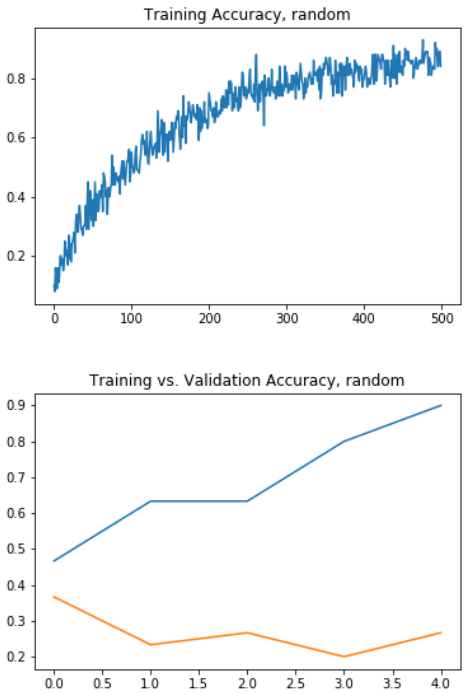

Fig. 6. Random Labels

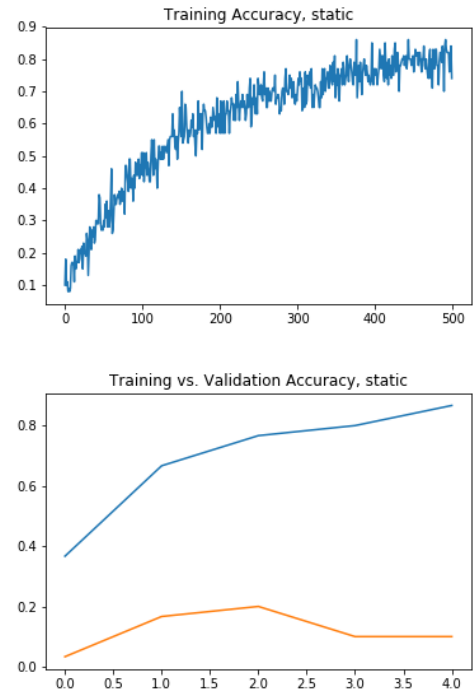

Fig. 7. Random Data (static)

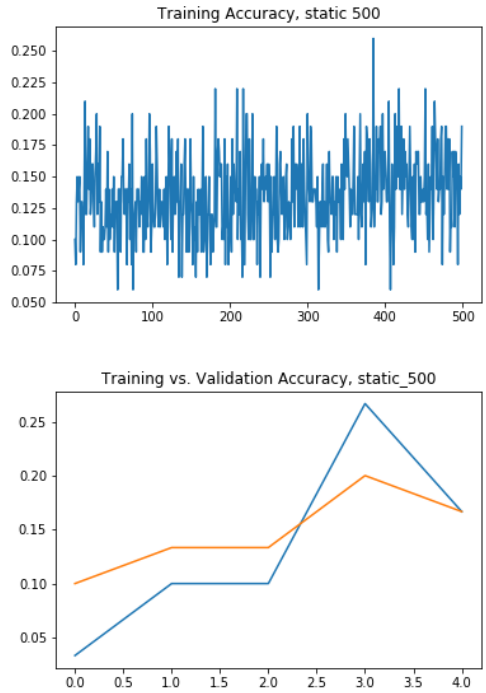

Fig. 8. Random Data (large training size)

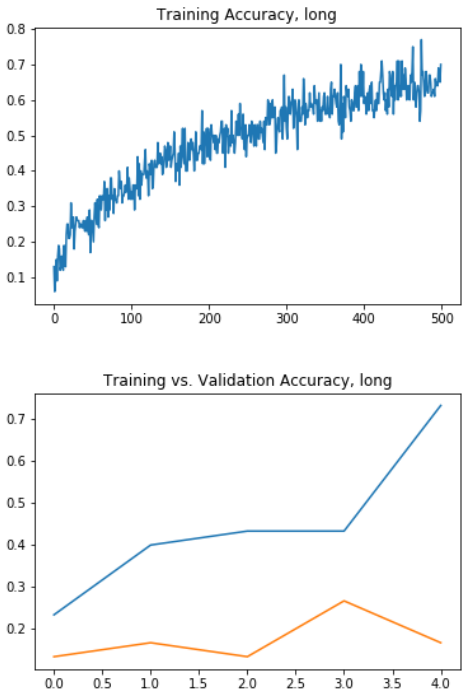

Fig. 9. Original Data (large training size and time)